TOPSTECHNOLOGIES
Training | Outsourcing | Placement | Study Abroad

ASSESSMENT FILE
SOFTWARE ENGINEERING
ASSESSMENT CODE : M5-A1
M5 - Database Management

Section A — Concept Application

1. **SCENARIO**

You are designing the database for a food delivery application. It stores restaurant details, menu items linked to each restaurant, and customer orders. You must decide which columns need NOT NULL constraints, which serve as PRIMARY KEYs, and which link tables via FOREIGN KEYs.

Question: Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

➔ In a food delivery database, constraints are important for maintaining data integrity.
For example, consider the restaurants and menu_items tables.

1. restaurant_id — PRIMARY KEY

The **restaurant_id** column should be a **PRIMARY KEY** because every restaurant must have a unique identifier.

```
restaurant_id INT PRIMARY KEY
```

This prevents two restaurants from having the same ID.

2. name — NOT NULL

The restaurant name should be NOT NULL because every restaurant record should have a name.

```
name VARCHAR(100) NOT NULL
```

Without this constraint, a restaurant could be stored without a name, making it difficult to identify.

3. restaurant_id in menu_items — FOREIGN KEY

The restaurant_id in menu_items should be a FOREIGN KEY referencing the restaurant table.

```
FOREIGN KEY (restaurant_id)
REFERENCES restaurants(restaurant_id)
```

This ensures that a menu item can only belong to an existing restaurant.

Problems without these constraints

- Without PRIMARY KEY, duplicate restaurant IDs could occur.
- Without NOT NULL, important information such as restaurant names could be missing.
- Without FOREIGN KEY, menu items could reference restaurants that do not exist, creating orphan records.

2. SCENARIO

You are managing the database for a food delivery platform. The operations team asks you to update the status of all orders placed more than 7 days ago that are still marked 'Pending' to 'Cancelled', and to permanently delete menu items marked 'Unavailable' from restaurants that have closed.

Question: Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition?

➔ The scenario asks us to update old pending orders and delete unavailable menu items from closed restaurants.

UPDATE example

```
UPDATE orders
SET status = 'Cancelled'
WHERE order_date < DATE_SUB(CURDATE(), INTERVAL 7 DAY)
  AND status = 'Pending';
```

The **WHERE** clause is essential because it identifies **only the orders that satisfy the required conditions**.

```
DELETE example
DELETE FROM menu_items
WHERE status = 'Unavailable'
  AND restaurant_id IN (
     SELECT restaurant_id
     FROM restaurants
     WHERE status = 'Closed'
  );
```

The **WHERE** clause prevents all menu items from being deleted.

What happens without WHERE?

For example:

```
UPDATE orders
SET status = 'Cancelled';
```

This would change **every order** to **Cancelled**.

Similarly:

```
DELETE FROM menu_items;
```

would permanently delete **all menu items** from the table.

Therefore, **WHERE** is critical for safely restricting DML operations to the intended records.

3. SCENARIO

You are building a revenue dashboard for a food delivery startup. The management wants total revenue, average order value, and number of orders per restaurant — but only for restaurants that have processed more than 20 orders in the current month.

Question: Which SQL clauses and aggregate functions would you use to produce this report?

Explain why HAVING must be used here rather than WHERE to filter by aggregate results.

➔ The management wants total revenue, average order value and number of orders per restaurant, but only for restaurants with more than 20 orders.

We can use:

- SUM() → total revenue
- AVG() → average order value
- COUNT() → number of orders
- GROUP BY → group orders by restaurant
- HAVING → filter grouped/aggregate results

Example:

```
SELECT
   restaurant_id,
   COUNT(order_id) AS total_orders,
   SUM(total_amount) AS total_revenue,
   AVG(total_amount) AS average_order_value
FROM orders
WHERE MONTH(order_date) = MONTH(CURDATE())
  AND YEAR(order_date) = YEAR(CURDATE())
GROUP BY restaurant_id
HAVING COUNT(order_id) > 20;
```

Why HAVING instead of WHERE?

WHERE filters individual rows **before grouping**.

HAVING filters groups **after aggregate functions have been calculated**.

Therefore:

HAVING COUNT(order_id) > 20

is required because **COUNT()** is an aggregate function.

4. SCENARIO

You are developing an order history page. The orders table has order_id, customer_id, restaurant_id, and total_amount. The restaurants table has restaurant_id and name. The page must show order ID, restaurant name, and amount for every order — including any that reference a restaurant no longer in the table.

Question: Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why?

Describe what each join type does to records that have no match in the other table.

➔ The order history page must show every order, including orders whose restaurant no longer exists.

INNER JOIN

```
SELECT
   o.order_id,
   r.name AS restaurant_name,
   o.total_amount
FROM orders o
INNER JOIN restaurants r
   ON o.restaurant_id = r.restaurant_id;
```

INNER JOIN returns only records where a matching restaurant exists.

If an order refers to a restaurant that has been removed, that order will **not appear**.

LEFT JOIN

```
SELECT
    o.order_id,
    r.name AS restaurant_name,
    o.total_amount
FROM orders o
LEFT JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id;
```

LEFT JOIN returns **all orders** from the **orders** table.

If there is no matching restaurant, restaurant_name will contain NULL.

Which one should be used?

LEFT JOIN should be used because the requirement says that every order must be displayed, even when its restaurant is no longer available.

5. **SCENARIO**

You are building the analytics layer for a food delivery platform. A senior developer proposes a VIEW called top_restaurants showing restaurant name, total orders, and average rating — hiding raw ID columns from the front-end team.

Question: Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

➔ A VIEW named **top_restaurants** can hide unnecessary raw database details from the front-end team.

For example:

```
CREATE VIEW top_restaurants AS
SELECT
   r.name AS restaurant_name,
   COUNT(o.order_id) AS total_orders,
   AVG(o.rating) AS average_rating
FROM restaurants r
JOIN orders o
   ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.name;
```

Why use a VIEW?

1. **Security —** raw IDs or sensitive columns can be hidden.
2. **Simplicity —** the front-end developer can query the view instead of writing complicated joins.
3. **Reusability —** the same query can be reused.
4. **Abstraction —** the underlying table structure is hidden from the front-end.

Two limitations

1. A VIEW may have limited ability to perform **INSERT**, **UPDATE**, or **DELETE**, especially when it contains joins or aggregate functions.
2. A VIEW does not normally store its own independent data; the results are based on the underlying tables.

6. **SCENARIO**

You are implementing the payment module for a food delivery app. When a customer places an order, the system must deduct the amount from the customer's wallet balance and record the payment simultaneously. During load testing, the wallet deduction completes but the payment record insert occasionally fails, leaving balances inconsistent.

Question: Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect.

➔ The problem occurs because the wallet deduction succeeds while the payment insertion fails.

The correct mechanism is a **database transaction**.

The transaction should make both operations succeed together or fail together.

COMMIT

COMMIT permanently saves all changes made during the transaction.

ROLLBACK

ROLLBACK cancels changes made during the transaction.

SAVEPOINT

SAVEPOINT creates a temporary point inside a transaction to which we can roll back without cancelling the entire transaction.

Example:

```
START TRANSACTION;

UPDATE customers
SET wallet_balance = wallet_balance - 500
WHERE customer_id = 1;

SAVEPOINT wallet_updated;

INSERT INTO payments(customer_id, amount)
VALUES (1, 500);

COMMIT;
```

If payment insertion fails:

```
ROLLBACK TO SAVEPOINT wallet_updated;
```

Or, if the complete transaction should be cancelled:

```
ROLLBACK;
```

This prevents the wallet from being permanently deducted when the payment record was not successfully created.

SECTION D — AI-Augmented Learning

1. **The exact prompt(s) my gave the AI tool.**

➔ Create a MySQL 8.x-compatible stored procedure named restaurant_monthly_summary.
The procedure should accept restaurant_id and month_number (1 to 12) as input parameters.

It should return:
1. Total number of orders
2. Total revenue
3. Average order value

for the specified restaurant and month.

Use SELECT, GROUP BY and aggregate functions.

If there are no orders for the specified restaurant and month, return a meaningful message instead of an empty result.

Make sure the procedure works in MySQL Workbench and XAMPP
MySQL/MariaDB.

2. The AI's original code and my corrected version.

➔ AI's Original Code

```
DELIMITER $$

CREATE PROCEDURE monthly_summary(
   IN p_restaurant_id INT,
   IN p_month INT
)
BEGIN

   SELECT
      COUNT(order_id) AS total_orders,
      SUM(total_amount) AS total_revenue,
      AVG(total_amount) AS average_order_value
   FROM orders
   WHERE restaurant_id = p_restaurant_id
     AND MONTH(order_date) = p_month
   GROUP BY restaurant_id;

END$$

DELIMITER ;
```

Problem with this version

If there are no orders for the selected restaurant/month, the GROUP BY query may return no row at all.

The assignment specifically asks for a meaningful message when there are no orders.

So we can improve it.

Corrected Version

```
DELIMITER $$

CREATE PROCEDURE restaurant_monthly_summary(
   IN p_restaurant_id INT,
   IN p_month INT
)
BEGIN

   DECLARE v_order_count INT DEFAULT 0;

   -- Validate month
   IF p_month < 1 OR p_month > 12 THEN

      SELECT
         'Invalid month. Month must be between 1 and 12.' AS message;

   ELSE

      -- Check whether orders exist
      SELECT COUNT(*)
      INTO v_order_count
      FROM orders
      WHERE restaurant_id = p_restaurant_id
        AND MONTH(order_date) = p_month;

      IF v_order_count = 0 THEN

         SELECT
            'No orders found for this restaurant in the selected month.'
            AS message;

      ELSE

         SELECT
```

```
            restaurant_id,
            COUNT(order_id) AS total_orders,
            SUM(total_amount) AS total_revenue,
            AVG(total_amount) AS average_order_value
        FROM orders
        WHERE restaurant_id = p_restaurant_id
          AND MONTH(order_date) = p_month
        GROUP BY restaurant_id;

    END IF;

  END IF;

END$$

DELIMITER ;
```

3. **A 3–4 line note explaining what you changed and why the AI's original version needed the correction.**

➔ The original AI procedure returned an empty result when there were no orders for the selected restaurant and month. I improved the procedure by first checking the number of matching orders using COUNT(*). I also added month validation to ensure that only values from 1 to 12 are accepted. This makes the procedure more reliable and provides meaningful messages to the user.